

MONISHRAJ S

2462518

5 BT CS IOT

CIA – 2

27/08/2026

PRATICAL

Task 1 — Node.js Development Setup & Running Node.js Files

Create a new project and initialize npm:

```
mkdir countdown-app  
cd countdown-app  
npm init -y
```

Create countdown.js:

```
console.log("Countdown App Ready");
```

Run:

```
node countdown.js
```

Exact output:

Countdown App Ready

Task 2

```
console.log("Countdown App Started");
```

```
setTimeout(() => {  
    console.log("Timer finished!");  
}, 3000);
```

```
console.log("You can still enter terminal input while the timer runs.");
```

Run:

```
node countdown.js
```

Exact output:

Countdown App Started

You can still enter terminal input while the timer runs.

Timer finished!

Task 3

```
const { setInterval, clearInterval } = require("timers");
```

```
let seconds = 5;
```

```
const timer = setInterval(() => {  
  console.log(`Remaining: ${seconds}`);  
  
  seconds--;  
  
  if (seconds < 0) {  
    clearInterval(timer);  
    console.log("Countdown finished!");  
  }  
}, 1000);
```

Run:

```
node countdown.js
```

Exact output:

```
Remaining: 5  
Remaining: 4  
Remaining: 3  
Remaining: 2  
Remaining: 1  
Remaining: 0  
Countdown finished!
```

Task 4

```
const start = new Date("2026-08-27T12:00:00");  
const end = new Date("2026-08-27T12:00:10");  
const seconds = (end - start) / 1000;  
console.log(seconds);
```

Exact output:

```
10
```

```
const start = new Date("2026-08-27T12:00:00");  
const end = new Date("2026-08-27T12:00:10");  
  
const seconds = (end - start) / 1000;
```

```
console.log(`Seconds remaining: ${seconds}`);
```

Run:

```
node countdown.js
```

Output:

```
Seconds remaining: 10
```

Task 5

```
let seconds = parseInt(process.argv[2]);

if (isNaN(seconds) || seconds <= 0) {
  console.log("Please provide a valid number of seconds.");
  process.exit();
}

console.log(`Countdown started for ${seconds} seconds.`);
console.log('Type "cancel" to stop the countdown.');
```

```
const timer = setInterval(() => {
  console.log(`Remaining: ${seconds}`);

  seconds--;

  if (seconds < 0) {
    clearInterval(timer);
    console.log("Countdown finished!");
    process.exit();
  }
}, 1000);

process.stdin.setEncoding("utf8");

process.stdin.on("data", (input) => {
  if (input.trim().toLowerCase() === "cancel") {
    clearInterval(timer);
    console.log("Countdown cancelled!");
    process.exit();
  }
});
```

Run:

```
node countdown.js 5
```

Exact output if not cancelled:

```
Countdown started for 5 seconds.  
Type "cancel" to stop the countdown.  
Remaining: 5  
Remaining: 4  
Remaining: 3  
Remaining: 2  
Remaining: 1  
Remaining: 0  
Countdown finished!  
  
Countdown cancelled!
```

Task 6

```
npm install --save-dev nodemon
```

```
{  
  "scripts": {  
    "dev": "nodemon countdown.js"  
  }  
}
```

For demonstration, countdown.js can contain:

```
console.log("Countdown App Started");
```

Run:

```
npm run dev
```

You will see NodeMon startup text plus:

```
Countdown App Started
```

Change the message to:

```
console.log("Countdown App Updated");
```

Save the file. NodeMon restarts the app and prints:

```
Countdown App Updated
```

Task 7

```
let seconds = 5;  
  
const timer = setInterval(() => {  
  console.log(`Remaining: ${seconds}`);  
  
  seconds--;
```

```
// BUG: Wrong comparison operator
if (seconds > 0) {
  clearInterval(timer);
  console.log("Countdown finished!");
}
}, 1000);
```

Run:

```
node countdown.js
```

Buggy output:

```
Remaining: 5
Countdown finished!
```

The bug is the condition `seconds > 0`. After decrementing, `seconds` becomes 4, so the condition is immediately true.

Fix it by changing:

```
if (seconds > 0)
```

to:

```
if (seconds < 0)
```

Correct code:

```
let seconds = 5;

const timer = setInterval(() => {
  console.log(`Remaining: ${seconds}`);

  seconds--;

  if (seconds < 0) {
    clearInterval(timer);
    console.log("Countdown finished!");
  }
}, 1000);
```

```
// Bug fixed: changed > to < so the countdown continues until zero.
```

Exact output:

```
Remaining: 5
Remaining: 4
Remaining: 3
Remaining: 2
Remaining: 1
```

Remaining: 0
Countdown finished!

Task 8

```
function checkTimeLeftCallback(seconds, callback) {
  setTimeout(() => {
    const remaining = seconds - 1;
    callback(remaining);
  }, 1000);
}

checkTimeLeftCallback(5, (remaining) => {
  console.log(`Time remaining: ${remaining} seconds`);
});
```

Run:

```
node countdown.js
```

Exact output after about 1 second:

```
Time remaining: 4 seconds
```

Full countdown version:

```
function checkTimeLeftCallback(seconds, callback) {
  setTimeout(() => {
    callback(seconds - 1);
  }, 1000);
}

let seconds = 5;

function countdown() {
  checkTimeLeftCallback(seconds, (remaining) => {
    console.log(`Remaining: ${remaining}`);

    if (remaining > 0) {
      seconds = remaining;
      countdown();
    } else {
      console.log("Countdown finished!");
    }
  });
}

countdown();
```

Full countdown output:

```
Remaining: 4
Remaining: 3
Remaining: 2
Remaining: 1
Remaining: 0
Countdown finished!
```

Task 9

```
let seconds = 5;

const timer = setInterval(() => {
  console.log(`Remaining: ${seconds}`);

  if (seconds === 0) {
    clearInterval(timer);

    setTimeout(() => {
      console.log("Time's up!");
    }, 0);
  }

  seconds--;
}, 1000);
```

Run:

```
node countdown.js
```

Exact output:

```
Remaining: 5
Remaining: 4
Remaining: 3
Remaining: 2
Remaining: 1
Remaining: 0
Time's up!
```

Task 10

```
function checkTimeLeftPromise(seconds) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (seconds > 0) {
        resolve(seconds - 1);
      } else {
        reject("Invalid time");
      }
    }, 1000);
  });
}
```

```
    });  
  }  
  
  checkTimeLeftPromise(5)  
    .then((remaining) => {  
      console.log(`Time remaining: ${remaining} seconds`);  
    })  
    .catch((error) => {  
      console.log("Error:", error);  
    });  
}
```

Run:

```
node countdown.js
```

Exact output:

```
Time remaining: 4 seconds
```

Output:

```
Error: Invalid time
```

\

THEORY

MONISHRAJ A

2462519

5 BT CS 10T

27/08/2024

CIA-2

Theory

(Q1)

Running node .app.js when app.js doesn't exist

Node.js reports:

Error: Cannot find module './app.js'

Fix: Create app.js in the current folder.

(Q2) Role of V8 and libuv

V8 executes JavaScript code libuv handles asynchronous operations such as network & file I/O using the event loop

(Q3) Node.js js documents.

The js documents show available methods, parameter return values, callback behavior, it helps choose synchronous methods

(Q4)

.exit vs ctrl + C twice

.exit: Explicitly exit the Node.js REPL

Ctrl + C twice: Forces the REPL / process to terminate

(Qr) process.argv

output: node app.js hello world

⇒ array contain

```
[  
  node-path,  
  app.js-path,  
  "hello",  
  "world"  
]
```

(Q6) when NOT to use Node Mon

Node Mon should not be used in production, because it is mainly a development tool that automatically restarts the application when files change.

(Q11) Exact output: caught: err.
await. Promise.reject('err') throws an error, which is caught by catch.

(Q12) a, b, c ⇒ a() calls b() synchronously, so b is print before c

(Q9) SetTimeout (0) vs setImmediate()

setImmediate(fn, 0) → runs in timer phase

setImmediate(fn) → runs in check phase after I/O call back

(Q10) function delay (ms) {

return new Promise (resolve => {

setTimeout (resolve, ms);

});

⇒ It resolves with no value after ms (milliseconds)